FULLY PARALLELIZED GPU-ONLY HIGH PERFORMANCE JPEG DECODER

Beste Güney, Carlos Fernández, Debeshee Das, Emmy Zhou, Nayanika Debnath

Department of Computer Science
ETH Zürich
Zürich, Switzerland

ABSTRACT

The JPEG format has been the standard for lossy image compression for several decades, offering high compression ratios with minimal perceptual loss in image quality. In GPU-accelerated computer vision and deep learning workflows, such as training image classification models, efficient JPEG decoding is essential. However, most decoders are CPU-based, requiring the transfer of decoded image data to GPUs via interconnects like PCI-E. This significantly reduces throughput, making JPEG decoding a critical bottleneck in such pipelines. In contrast, a GPU-accelerated decoder can enhance efficiency by transferring only compressed data, with parallelized decoding performed directly on the accelerator. Despite its potential, parallel JPEG decoding poses challenges due to the strictly serial nature of entropy-based compression. Building on optimizations from prior work, we propose a novel high-performance CUDA JPEG decoder that achieves high throughput by parallelizing all stages of the decoding process directly on the GPU. We obtain an *improvement in throughput over state-of-the-art* CPU (libjpeg) and GPU (nvJPEG) based decoders by 43% and 2.2% respectively on a AMD Ryzen 7 2700 CPU with 16 cores and an NVIDIA GeForce RTX 2080 Ti GPU. Our code can be found in our GitHub repository[1].

1. INTRODUCTION

Motivation. JPEG encoding is a lossy compression standard designed to reduce digital image sizes by exploiting the limitations of human visual perception. Most image datasets used in large-scale machine learning pipelines for tasks like image classification or generative AI are commonly stored in this compressed JPEG format. Consequently, GPU-accelerated computer vision and deep learning workflows require efficient decoding of JPEG images into expanded pixel arrays prior to processing or model training. However, many existing decoders are CPU-based, necessitating the transfer of decoded image data to GPUs via interconnects such as PCI-E. This significantly reduces throughput, making JPEG decoding a critical bottleneck in these pipelines. In contrast, a GPU-accelerated decoder can improve efficiency by transferring only compressed data, with decoding performed in parallel directly on the GPU. Although some stages of the algorithm are well-suited for massive GPU parallelism, the inherently serial design of its entropy-based compression makes it very difficult to fully parallelize the process.

Key Contributions. We propose a novel high-performance CUDA JPEG decoder that achieves high throughput by decoding images directly on the GPU, leveraging various optimizations. Our decoder demonstrates a throughput improvement of 43% over state-of-the-art CPU-based decoders (libjpeg) and 2.2% over GPU-based decoders (nvJPEG). We conduct a comprehensive benchmarking study, comparing throughput and runtime performance against several high-performance decoders, including zune-jpeg [1], jpeglib [2], and nvJPEG [3]. We provide detailed compute and memory profiling, along with an in-depth analysis of the effects of batch sizes and thread allocation per image on throughput.

[1] https://github.com/debesheedas/GPU-JPEG-Decoder

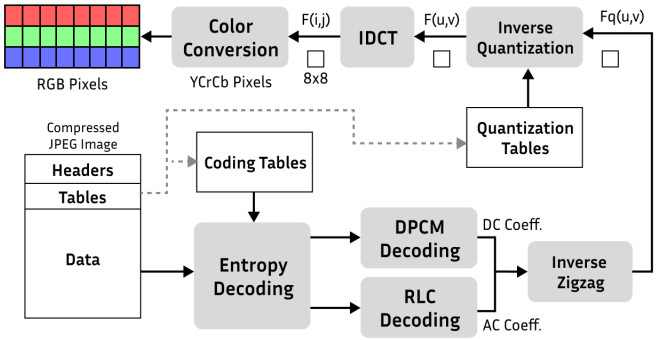

Fig. 1. JPEG Decoding Algorithm Flowchart

2. BACKGROUND: JPEG ENCODING AND DECODING

Encoding. The encoding pipeline begins with a color space conversion from RGB to YCrCb, where the luminance (Y) and chrominance (Cr and Cb) components are processed independently. Chrominance components are often downsampled to take advantage of the human eye's reduced sensitivity to color differences. Our decoder supports only JPEG images with 4:4:4 subsampling, the highest resolution compression format, which avoids downsampling at this stage. Next, the image is divided into 8×8 blocks, and each block undergoes a Discrete Cosine Transform (DCT) to convert spatial domain information into frequency components. The resulting coefficients are quantized using predefined quantization tables, reducing precision in high-frequency components to achieve compression. These quantized coefficients are then arranged in a zigzag order to prioritize low-frequency components, which are more important for visual quality, in the upper-left corner of the 8x8 matrix. Finally, the coefficients are encoded using entropy coding techniques, such as Huffman or Arithmetic coding.

Decoding. The JPEG decoding pipeline (Fig. 1) is structured to systematically reverse the encoding process. The following steps outline the technical sequence involved in decoding the compressed JPEG image:

1. **Entropy Decoding:** The compressed bitstream is parsed and decoded using Huffman tables extracted from the JPEG file's header. This process recovers both DC and AC coefficients for each 8×8 block across the three color channels (Y, Cr, and Cb). DC coefficients represent the average color in an 8×8 block and are encoded using Differential Pulse Code Modulation (DPCM). Instead of encoding the DC value directly, the difference between the current and previous block's DC coefficient is stored. The decoded DPCM value is added to the previous block's DC value to restore the original DC coefficient. AC coefficients, which correspond to variations in pixel values within the block, are encoded using Run-Length Encoding (RLE), where consecutive zeros are paired with a non-zero value in the form (RunLength, Value). For instance, a long sequence of zeros might be represented as (20, Value), where "20" indicates the number of zeros, and "Value" is the actual non-zero AC coefficient. This method reduces the amount of data to be stored, as high-frequency components often contain many zeros.
2. **Inverse Zigzag:** The decoded coefficients are rearranged from their zigzag order back into the standard 8x8 matrix format using a fixed mapping matrix.
3. **Inverse Quantization:** The decoded coefficients are scaled back to approximate their original values by multiplying each coefficient by the corresponding value from the quantization table. This step only approximately reverses the quantization applied during encoding, which reduces the precision of the frequency domain coefficients, making JPEG encoding inherently lossy.
4. **Inverse Discrete Cosine Transform (IDCT):** The dequantized coefficients are transformed from the frequency domain back to the spatial domain to reconstruct the YCrCb pixels using IDCT.

$$f(x,y) = \frac{1}{4} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} A_{u,v}\, C(u)\, C(v)$$

$$\cdot \cos\left(\frac{\pi(2x+1)u}{2N}\right) \cos\left(\frac{\pi(2y+1)v}{2N}\right)$$

where:

$$C(u) = \begin{cases} \frac{1}{\sqrt{N}} & \text{if } u = 0, \\ \sqrt{\frac{2}{N}} & \text{if } u > 0, \end{cases}$$

and similarly for $C(v)$. Here, $f(x, y)$ are the spatial pixel values of the decoded block, and $A_{u,v}$ are the DCT coefficients of size $N \times N$.

5. **Color Conversion:** The reconstructed YCrCb pixel values are then converted to RGB pixel values using a linear transformation completing the decoding pipeline.

3. RELATED WORK

Early GPU-based solutions [4, 5] offloaded only the IDCT stage to the GPU, with other stages remaining on the CPU. These decoders employed asynchronous CPU-GPU decoding, achieving significant acceleration over CPU-only decoders. Other stages, such as zigzag, dequantization, IDCT, and color conversion, can be easily parallelized on the GPU. However, parallelizing entropy decoding is challenging due to the unknown block boundaries prior to this stage.

The 'JParEnt' method [6] addresses this by using a fast CPU scan to approximate block boundaries for parallel decoding. Wang et al. [7] proposed two pre-scan methods to reduce repetitive boundary scans on the CPU, achieving a modest 1.5x speedup over nvJPEG. Many studies [8, 9, 10] have focused on exploiting Huffman coding properties to parallelize decoding, particularly for parallel Huffman decoding. Klein and Wiseman [11] developed a parallel algorithm for multicore processors based on Huffman's self-synchronizing property, which has been used to implement a fully GPU-based Huffman decoder [12]. Building on this, [13] proposed a GPU-accelerated JPEG decompression method, achieving a 51x speedup over libjpeg-turbo and a 3.4x speedup over NVIDIA's nvJPEG, making it the most performant GPU-based JPEG decoder in the academic literature.

4. OUR HIGH PERFORMANCE CUDA JPEG-DECODER

In this section, we describe our high-performance CUDA JPEG decoder optimized for throughput, supporting both single-image and batch decoding with two dedicated kernels: `decodeKernel` for single images and `batchDecodeKernel` for batches. Both kernels invoke `decodeImage` (see Algorithm 1), which implements the decoding logic using appropriate memory pointers. For batch decoding, `batchDecodeKernel` is launched with the configuration `<<<batch_size, n_threads>>>`, where each image is assigned to a GPU thread block with `n_threads`, while the grid size matches the batch size.

Algorithm 1: `decodeImage` device side function to decode each image in the batch.

Input : Pointers to raw `imageData, quantTables, huffmanTables`
Output: `outputChannels` with decompressed image

in parallel Copy `zigzagMap` to shared memory;
`__syncthreads()`
`parallelHuffManDecode()`;
`__syncthreads()`
do in parallel
  while *pixelIndex* $< 3\times$ *totalPixels* **do**
    `performZigzagReordering(`*pixelIndex*`)`;
    pixelIndex $\longleftarrow$ *pixelIndex* + *numThreads*;
  end
end
`__syncthreads()`
do in parallel
  while *pixelIndex* $\times 8 < 3\times$ *totalPixels* **do**
    `idctRow(`*pixelIndex*`)`;
    pixelIndex $\longleftarrow$ *pixelIndex* + *numThreads*;
  end
end
do in parallel
  while *pixelIndex* $\times 8 < 3\times$ *totalPixels* **do**
    `idctCol(`*pixelIndex*`)`;
    pixelIndex $\longleftarrow$ *pixelIndex* + *numThreads*;
  end
end
`__syncthreads()`
`performColorConversion(outputChannels)`;

4.1. Extract and Allocate

We first parse the raw JPEG data on the CPU to extract information from the headers, including the quantization tables and Huffman trees. The trees are built using the classic Huffman coding algorithm, where the frequency of each character is determined by the length of its encoded bit stream. Less frequent characters are combined into internal nodes, and traversal of the tree appends 0 for left children and 1 for right children. The leaf node value corresponds to the bit stream constructed along the path. Finally, we allocate GPU memory and copy the compressed image data and Huffman lookup tables for the entire batch.

4.2. Parallelized Entropy Decoding

Entropy decoding is challenging to parallelize because the encoded data consists of variable-length bit sequences, requiring prior knowledge of their boundaries to divide the input into blocks for parallel execution. In our parallel Huffman decoding method, the bitstream is divided into multiple segments, each assigned to a separate thread for decoding. However, misalignment of segment boundaries with codeword boundaries can result in decoding errors if processed directly.

To overcome this issue, we have adopted the parallel Huffman decoding approach presented in [13], which takes advantage of the self-synchronising properties of Huffman codes. This property ensures that decoding errors self-correct after processing a certain number of bits. When a thread starts decoding its segment, it may initially produce erroneous symbols due to misaligned boundaries or incorrect Huffman table usage. The thread continues decoding into the adjacent segment, correcting any errors until a synchronization point is identified. This synchronization point indicates the moment when the decoding output matches that of the previous thread, ensuring both would produce identical results from that point onward. If a synchronization point is not found within the immediate overflow range, the thread extends further into subsequent segments to restore alignment. This overflow correction mechanism enables multiple threads to work in parallel while correcting decoding errors.

In leveraging the self-synchronising property of Huffman codes, we employ a modified version of the algorithm described in [13]. A global state array, `s_info`, is defined to store the end state of each segment after it has been decoded by a thread. Each entry in `s_info` consists of four values: `p` (the position in the bitstream), `c` (the current colour component), `z` (the current zig-zag index ranging from 0 to 63), and `n` (the number of symbols decoded in the segment). A synchronization point is reached when further decoding produces output identical to that of a previous thread. Thus, for a thread to stop decoding, its `p`, `c`, and `z` values must match those of a preceding thread.

Algorithm 2 provides pseudocode for decoding a single segment of the bitstream, while Algorithm 3 illustrates how the approach from [13] is adapted to our use case. Unlike [13], we use only a single block of threads per image. This means we do not further divide segments into subsegments. Instead, the segment length is determined by the number of threads available, as shown in line `3` of Algorithm 3.

In the initial stage of parallel decoding, each thread starts decoding the bitstream segment corresponding to its thread ID, with the resulting state stored in `s_info` (line 4-8 Algorithm 3). Subsequently, each thread performs at least one iteration in overflow mode, where it continues decoding into the next segment using the end state of the preceding segment as its starting state. This process repeats until each thread either reaches a synchronization point or the end of the bitstream (line 9-22 Algorithm 3).

This approach guarantees correctness, as thread 0 always starts from the beginning of the bitstream. If no synchronization point is detected, thread 0 will continue decoding across the entire stream, effectively reverting to a fully serial Huffman decoding process. In practice, synchronization points are almost always reached. For instance, with 32 threads, synchronization points were typically achieved after one or two overflow iterations.

Once all threads have identified a synchronization point, we perform an exclusive prefix sum over the `n` values stored in `s_info` for each segment (line 23 Algorithm 3). This operation adjusts the `n` values so they represent the exact positions in the output array where the decoded symbols should be written. Then, each thread performs a final decoding pass on its assigned segment in write mode, writing the decoded symbols to the output buffer (line 25-30 Algorithm 3).

To ensure correct differential decoding of the DC coefficients, we conclude the process by performing a prefix sum over the DC coefficients stored in the output array, resolving any missing differences introduced during the parallel decoding phase (line 32-34 Algorithm 3).

Algorithm 2: Decoding of a single segment adopted from [13]

```
1  function decodeSegment(i, start, end, overflow,
     write, out, sInfo)
2      (p, n, c, z) ⟵ (start, 0, Y, 0);
3      positionInOutput ⟵ 0;
4      if write then
5          positionInOutput ⟵ sInfo[i].n;
6      end
7      if overflow then
8          (p, c, z) ⟵ sInfo[i − 1];
9      end
10     while p < end do
11         (length, symbol, runLength) ⟵
             decodeNextSymbol();
12         p ⟵ p + length;
13         positionInOutput ⟵
             positionInOutput + runLength;
14         n ⟵ n + runLength;
15         z ⟵ z + runLength;
16         if write then
17             out[positionInOutput] = symbol;
18         end
19         positionInOutput ⟵ positionInOutput + 1;
20         n ⟵ n + 1;
21         z ⟵ z + 1;
22         if z ≥ 64 or symbol == EOB then
23             z ⟵ 0;
24             c ⟵ next component;
25         end
26     end
27     return (p, n, c, z);
```

Algorithm 3: Parallel Huffman decoding algorithm adopted from [13]

```
function parallelHuffManDecode()
    segmentSize ⟵ ⌈bitstreamLength ÷ n⌉;
    for each thread t_i ∈ {0, ..., n − 1} do in parallel
        segmentInd ⟵ t_i;
        start ⟵ segmentInd · segmentSize;
        end ⟵ min(start+segmentSize, bitstreamLength);
        synchronized ⟵ false;
        sInfo[segmentInd] ⟵ decodeSegment(segmentInd, start, end, false, false, out, sInfo);
        __syncthreads();
        ++segmentInd;
        while not synchronized and segmentInd ≤ n − 1 do
            start ⟵ segmentInd · segmentSize;
            end ⟵ min(start+segmentSize, bitstreamLength);
            (p, n, c, z) ⟵ decodeSegment(segmentInd, start, end, true, false, out, sInfo);
            if (p, c, z) == sInfo[segmentInd] then
                synchronized ⟵ true;
            end
            sInfo[segmentInd] ⟵ (p, n, c, z);
            ++segmentInd;
            __syncthreads();
        end
    end
if t_i = 0 compute exclusive prefix sum on sInfo.n;
__syncthreads();
for each thread t_i ∈ {0, ..., n − 1} do in parallel
    start ⟵ t_i · segmentSize;
    end ⟵ min(start+segmentSize, bitstreamLength);
    if t_i == 0 then
        decodeSegment(0, start, end, false, true, out, sInfo);
    else
        decodeSegment(t_i, start, end, true, true, out, sInfo);
    end
end
if t_i < 3 do in parallel
    reverse difference coding for all DC coefficients in component t_i in image;
end
```

4.3. Inverse Zigzag and Quantization

We combine the inverse zigzag rearrangement with inverse quantization, with all threads allocated to the image performing the operations in sync to avoid warp divergence. With N threads per image, each thread processes a pixel in parallel, incrementing by N after each set. The while loop processes the entire image in chunks of N pixels. In the method `performZigzagReordering`, we revert the zigzag mapping to a row-wise mapping using the `zigzaglocations` table stored in shared memory for better performance, and apply the corresponding quantization table value.

4.4. Fast Integer IDCT

The Fast Integer IDCT [14] eliminates all `for` loops by performing eight row operations followed by eight column operations on each 8×8 block. Similar to the inverse zigzag algorithm, all threads work in parallel, with each thread processing a full row or column instead of a pixel index. Synchronization between rows and columns are avoided by assigning each 8×8 block to the same warp for both `idctRow()` and `idctCol()`, which run without branching statements.

4.5. Color Conversion

All threads work in parallel, processing as many pixels as possible at a time. For each pixel, the three channels are processed sequentially. The third channel is data dependent on the first two so this method works well. We have fully decoded pixel values in the output channels at the end of this stage, ready to be written to any output file.

5. EXPERIMENTAL RESULTS

Experimental setup. All experiments were conducted on an AMD Ryzen 7 2700 Eight-Core Processor with 16 cores and a base clock speed of 3.2 GHz (no boost), 32 GB RAM, and an NVIDIA GeForce RTX 2080 Ti GPU with 11 GB VRAM. The system ran Linux Ubuntu 11.4.0 with GCC version 11.4.0 and CUDA version 12.5. Accurate runtime measurements were obtained using Google Benchmark [15] and CUDA event timers. Each measurement was repeated 10 times, with standard deviations appropriately reported. For our CUDA decoder, the timer began after transferring raw JPEG data to device memory and ended when the output was available in device memory. Thus, timings excluded data transfer to the GPU, GPU memory allocation, and deallocation operations. However, since the other decoders benchmarked employ either CPU-only or hybrid CPU-GPU approaches, their timings include the complete decoding process, including any GPU data transfer overhead.

Results. To evaluate the code's performance across various input sizes, we benchmark the runtime for images with different dimensions. A benchmarking dataset of 1,000 natural JPEG images was constructed, comprising 100 images for each square size, ranging from 200×200 to 2000×2000 pixels. Fig. 2 illustrates the average runtime performance (shaded region denotes the standard deviation) of our cpp and CUDA decoders, processing a single image at a time, compared to state-of-the-art JPEG decoders: jpeglib, zune-jpeg, and nvJPEG. The results indicate that standard CPU-based decoders, such as jpeglib, exhibit significantly better runtimes than GPU-based decoders (nvJPEG and our CUDA decoder) for single-image decoding. Notably, our CUDA decoder outperforms nvJPEG for most image sizes.

Fig. 3 presents our key result: the throughput of our CUDA decoder surpasses that of nvJPEG. For this analysis, we used a dataset of 3,000 images with dimensions of 400×400 pixels and selected the optimal resource settings for each decoder to maximize throughput. For jpeglib, all 16 CPU cores were utilized, yielding a mean throughput of 308 MB/sec. For nvJPEG, a batch size of 1,024 and 16 CPU cores were

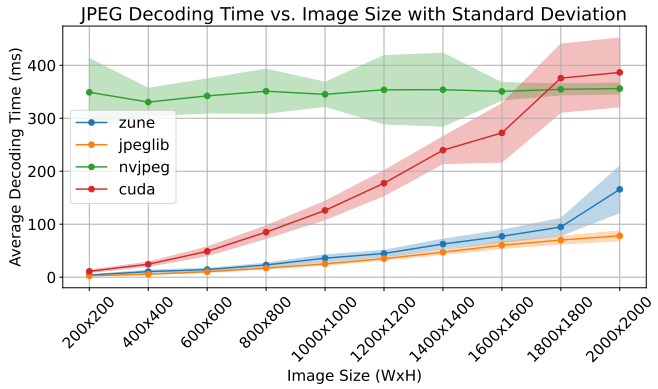

Fig. 2. Runtime Performance of our CUDA Decoder

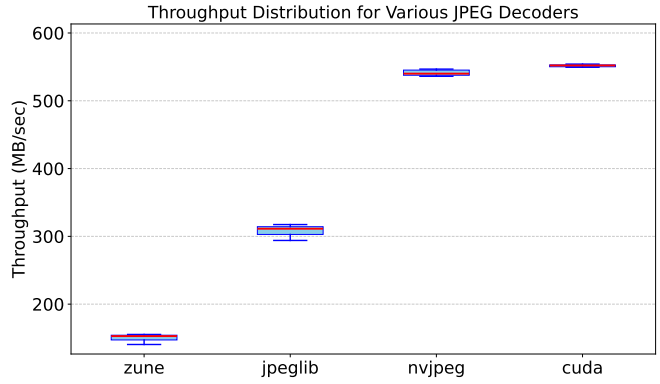

Fig. 3. Throughput Performance of our CUDA Decoder

used, resulting in a throughput of 540 MB/sec. In contrast, our CUDA decoder processed the entire dataset in a single batch, achieving a throughput of 552 MB/sec. To validate the statistical significance of this improvement, we performed an Analysis of Variance (ANOVA) [16]. Assuming the null hypothesis that the throughput of nvJPEG and our CUDA decoder is equivalent, we computed an F-statistic of 12.52 and a p-value of 0.0023 over our repeated measurements. These results confirm that the difference in throughput is statistically significant, leading to a rejection of the null hypothesis. Thus, our CUDA decoder demonstrates a statistically significant higher throughput compared to nvJPEG.

To observe the effect of batch sizes for GPU-based decoders on throughput, we report the following two plots using images of dimension 400x400: Fig. 4 takes a dataset of 3000 images and plots the throughput for various batch sizes ranging from 1 to 3000 in strides of powers of 2 and Fig. 5 shows the throughput trends for datasets of increasing sizes from 1 to 3000, but always processed in a single batch. The results show that nvJPEG performs well with increasing batch sizes up to 1,024, while our decoder continues to improve with larger batch sizes. Notably, when processing datasets in a single batch, our decoder consistently outperforms nvJPEG across all dataset sizes.

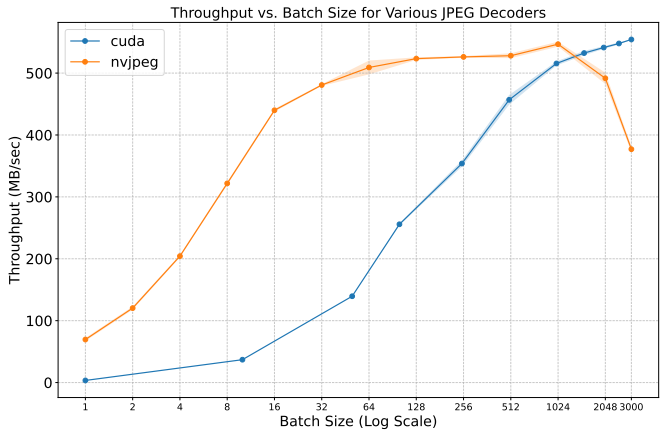

Fig. 4. Throughput Trends for Varying Batch Size

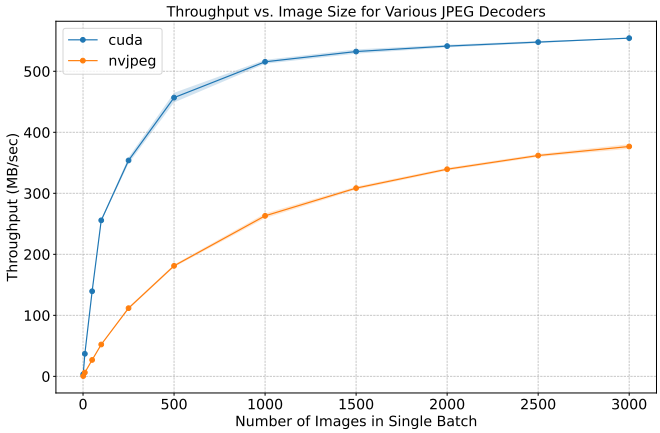

Fig. 5. Throughput for a Single Batch with Increasing Size

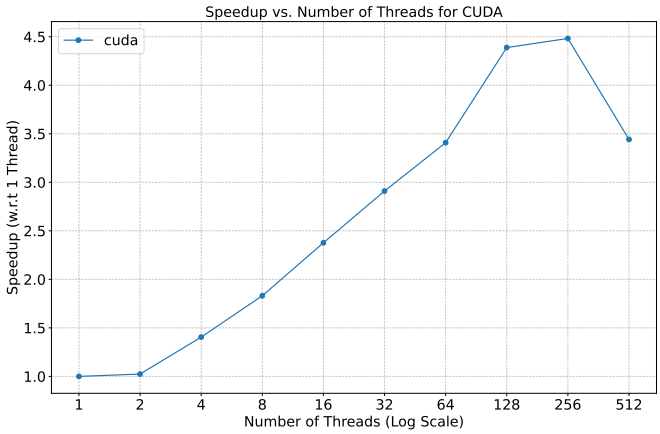

Fig. 6. Speedup (increase in throughput) by Varying Number of Threads per Image

To observe the throughput improvement with increasing threads per image, we plot the speedup relative to one thread per image in Fig. 6. A batch of 3000 images (400x400) is decoded, with the throughput for one thread being 110 MB/sec. The best speedup of $4.5\times$ is achieved with 256 threads per image. Throughput generally increases with more threads, leveraging the parallelized code, though performance drops beyond 256 threads, likely due to reaching the GPU's resource limits in a single kernel.

We use NVIDIA Nsight Compute to generate a profiling report and the roofline plot shown in Fig. 7. According to the plot, our code is memory bound, which is expected as the nature of the JPEG decoding algorithm is highly memory intensive. We obtain a single precision arithmetic intensity of 0.09 (green dot) and a double precision arithmetic intensity of 0.4 (orange dot). According to the report, the compute

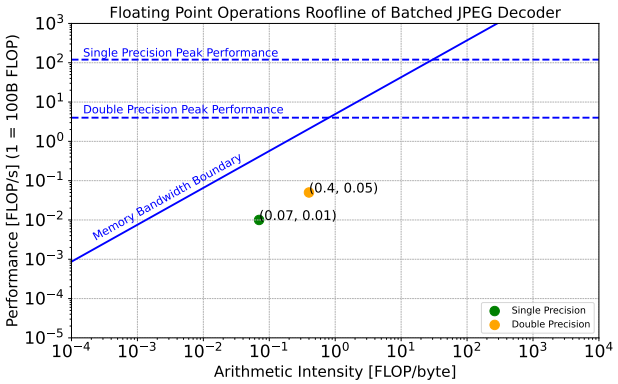

Fig. 7. Roofline Plot from NVIDIA NSight Compute Report

throughput obtained is about 45.9% and the memory throughput obtained is 44.9%. The memory workload profiling shows a high L1 cache hit rate of 98% and an L2 hit rate of 97.6%.

6. DISCUSSION

In this section, we explore several ideas we experimented with during the development of our high-performance JPEG decoder. While these concepts did not make it into the final implementation, they played a crucial role in shaping the design of the final version. We provide an overview of the 'legacy versions' of our decoder: *cudaH*, *cudaO*, *cudaS*, and *cudaB*.

cudaH, or cuda-hybrid, was inspired by early CUDA decoders where entropy decoding was performed serially (and efficiently) on the CPU. The decompressed data was then transferred to the GPU, where the remaining decoding steps were executed in parallel. However, the throughput of this hybrid approach was suboptimal because it involved transferring the decompressed image data (instead of the compressed data) from host to device memory, which added significant overhead. Despite this, *cudaH* delivered the best runtime performance on a single image compared to all our CUDA decoders and even outperformed nvJPEG by 49.5x, despite the inclusion of host-to-device memory transfer time.

With *cudaO*, we aimed to decode entirely on the GPU. We moved the raw JPEG bitstreams to the GPU, performing entropy decoding serially with a single thread, while parallelizing the remaining steps. To improve throughput, we attempted to pipeline the algorithm's stages, utilizing two warps. The first warp handled image block decoding, while the second handled IDCT and color conversion. We doubled the input buffer size, alternating work on different parts of the buffer between the two warps. Despite the theoretical promise, the long stalls caused by serial entropy decoding hindered performance gains, and we did not achieve a throughput improvement over the simpler approach of decoding images in parallel with a single thread per image.

All of our data and hence device-side memory operations are in global memory. To utilize the efficiency of shared memory, in *cudaS*, we moved chunks of data to shared memory, processed it there and then copied the final part back to global memory, all threads working in parallel. We also found out from our profiling the code using NVIDIA NSight Compute that we had many uncoalesced memory accesses between the L1 and L2 cache. To address this, we transposed the data from column-wise to row-wise when copying from global to shared memory for the IDCT column operations so that they would process contiguous blocks of data instead of the stride accesses before. Unfortunately, this did not improve our throughput much either. We speculate that the overhead of increased copying operations almost offset the gain from using shared memory and better access patterns.

At this point, the major performance bottleneck was the entropy decoding which was still being performed serially on the GPU (which is much slower than performing it serially on the CPU). We made a first attempt at parallelizing this in *cudaB* by decoding each block serially but attempting to decode all three channels of the block parallely instead of serially, theoretically expecting a speedup of 3x. We did this by making multiple educated guesses about where the second and third channels begin and decoded them parallelly with the first channel. When the first channel finished decoding, based on its position in the bit stream we could immediately know where the second channel should have started decoding and if any of our guesses were right. If we guessed right, we could also identify if the third channel was correctly decoded. If none of the guesses were right, we would just decode the remaining channels serially before moving on to the next block. Despite achieving high guess accuracies of 93% for the second channel and 70% for the third, overall throughput did not improve. We believe this was due to the high synchronization overhead (four syncs per block), making the parallel approach less efficient than simply decoding all channels serially.

7. CONCLUSION AND FUTURE WORK

For many GPU-accelerated computer vision and deep learning tasks, efficient JPEG decoding is essential due to limitations in memory bandwidth. Previous approaches to high performance JPEG decoding to this problem have not achieved high efficiency since they often employ hybrid designs where important parts of the decoding pipeline are still executed on the CPU. In this paper, we have presented an approach to parallel JPEG decoding entirely on GPUs based on the self-synchronizing property of Huffman codes, fast integer IDCT and other parallel optimizations. We implemented our algorithm using CUDA and benchmarked our runtime for single images, throughput for batches of images, and throughput trends with various batch sizes and number of threads on a NVIDIA GeForce RTX 2080 Ti GPU. We out-perform jpeglib by 224 MB/sec (43%) and nvJPEG by 12 MB/sec (2.2%) in throughput.

During development, we identified thread synchronisation and warp divergence as key performance bottlenecks and prioritised minimising their impact. We also focused on efficient memory access patterns and resolved memory leaks. While the algorithm remains memory-bound due to its intensive nature, it achieves high parallelisation without major computational bottlenecks.

Future improvements could focus on dynamically allocating the number of threads based on batch size, as the current static configuration can lead to inefficiencies across varying workloads. For smaller batch sizes, assigning more threads per image could enhance GPU resource utilisation and boost overall performance. Furthermore, refining memory access patterns to improve latency and making more effective use of shared memory could yield additional gains. Together, these refinements would strengthen the decoder's ability in handling diverse workloads with improved performance.

8. REFERENCES

[1] "https://docs.rs/zune-jpeg/latest/zune_jpeg/,".

[2] "https://libjpeg.sourceforge.net/,".

[3] "https://developer.nvidia.com/nvjpeg," .

[4] Ke Yan, Junming Shan, and Eryan Yang, "Cuda-based acceleration of the jpeg decoder," in *2013 Ninth International Conference on Natural Computation (ICNC)*. IEEE, 2013, pp. 1319–1323.

[5] Rushikesh Tade and Saniya Ansari, "Acceleration of jpeg decoding process using cuda," *International Journal of Computer Applications*, vol. 975, pp. 8887, 2015.

[6] Wasuwee Sodsong, Minyoung Jung, Jinwoo Park, and Bernd Burgstaller, "Jparent: Parallel entropy decoding for jpeg decompression on heterogeneous multicore architectures," in *Proceedings of the 7th International Workshop on Programming Models and Applications for Multicores and Manycores*, 2016, pp. 104–113.

[7] Lipeng Wang, Qiong Luo, and Shengen Yan, "Accelerating deep learning tasks with optimized gpu-assisted image decoding," in *2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2020, pp. 274–281.

[8] Evangelia Sitaridi, Rene Mueller, Tim Kaldewey, Guy Lohman, and Kenneth A Ross, "Massively-parallel lossless data decompression," in *2016 45th International Conference on Parallel Processing (ICPP)*. IEEE, 2016, pp. 242–247.

[9] Ritesh A Patel, Yao Zhang, Jason Mak, Andrew Davidson, and John D Owens, *Parallel lossless data compression on the GPU*, IEEE, 2012.

[10] Jiannan Tian, Cody Rivera, Sheng Di, Jieyang Chen, Xin Liang, Dingwen Tao, and Franck Cappello, "Revisiting huffman coding: Toward extreme performance on modern gpu architectures," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 881–891.

[11] Shmuel Tomi Klein and Yair Wiseman, "Parallel huffman decoding with applications to jpeg files," *The Computer Journal*, vol. 46, no. 5, pp. 487–497, 2003.

[12] André Weißenberger and Bertil Schmidt, "Massively parallel huffman decoding on gpus," in *Proceedings of the 47th International Conference on Parallel Processing*, 2018, pp. 1–10.

[13] André Weißenberger and Bertil Schmidt, "Accelerating jpeg decompression on gpus," in *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 2021, pp. 121–130.

[14] "https://github.com/mozilla/mozjpeg/blob/9b8d11f05e3ae4541ce5251f0e5c20c4cb8733b7/jidctfst.c#L171,".

[15] "https://github.com/google/benchmark/tree/main,".

[16] Ronald Aylmer Fisher et al., "014: On the" probable error" of a coefficient of correlation deduced from a small sample.," 1921.